RAJALAKSHMI ENGINEERING COLLEGE

An Autonomous Institution
Affiliated to Anna University, Chennai,
Rajalakshmi Nagar, Thandalam – 602 105

BONAFIDE CERTIFICATE

Name: .....JAYAPRIYA..B.....

Academic Year: 2025-2026 **Semester:** ..... **Branch:** .....

Register No. 211624170102 1

Certified that this is the bonafide record of work done by the above student in the..... *Laboratory during the academic year* 2025-2026

21/10/26

Signature of Faculty in-charge

Submitted for the Practical Examination held on.....

Internal Examiner **External Examiner**

HOME PAGE

AM Jayapriya B CLASS / SEC. CSD SUB.

S. NO.	Date	Description	Page No.
1.		Bresenham's line algorithm, Midpoint circle Algorithm, Ellipse Algorithm.	
2.		Implement the scan conversion of polygon & use flood fill algorithm to fill areas	
3.		2D Transformations (triangle, rectangle)	
4.		Clips Polygons to windows & display the clipped polygon.	
5.		3D Transformations (cube, pyramid)	
6.		Create & Render 3D objects using polygons & Apply basic colors & shading.	
7		Multimedia applications integrate images, sound, video.	
8.		Capture Video/Audio from Webcam/ microphone & display.	
9.		Create an application to send & receive messages.	
10.		Create a 3D object in Blender	

Completed 21/6/24

24/7/26

Expt. No: 1

Bresenham's Algorithm (Line)

Aim:

To implement the bresenham line drawing algorithm in python to generate 2 given points.

Program:

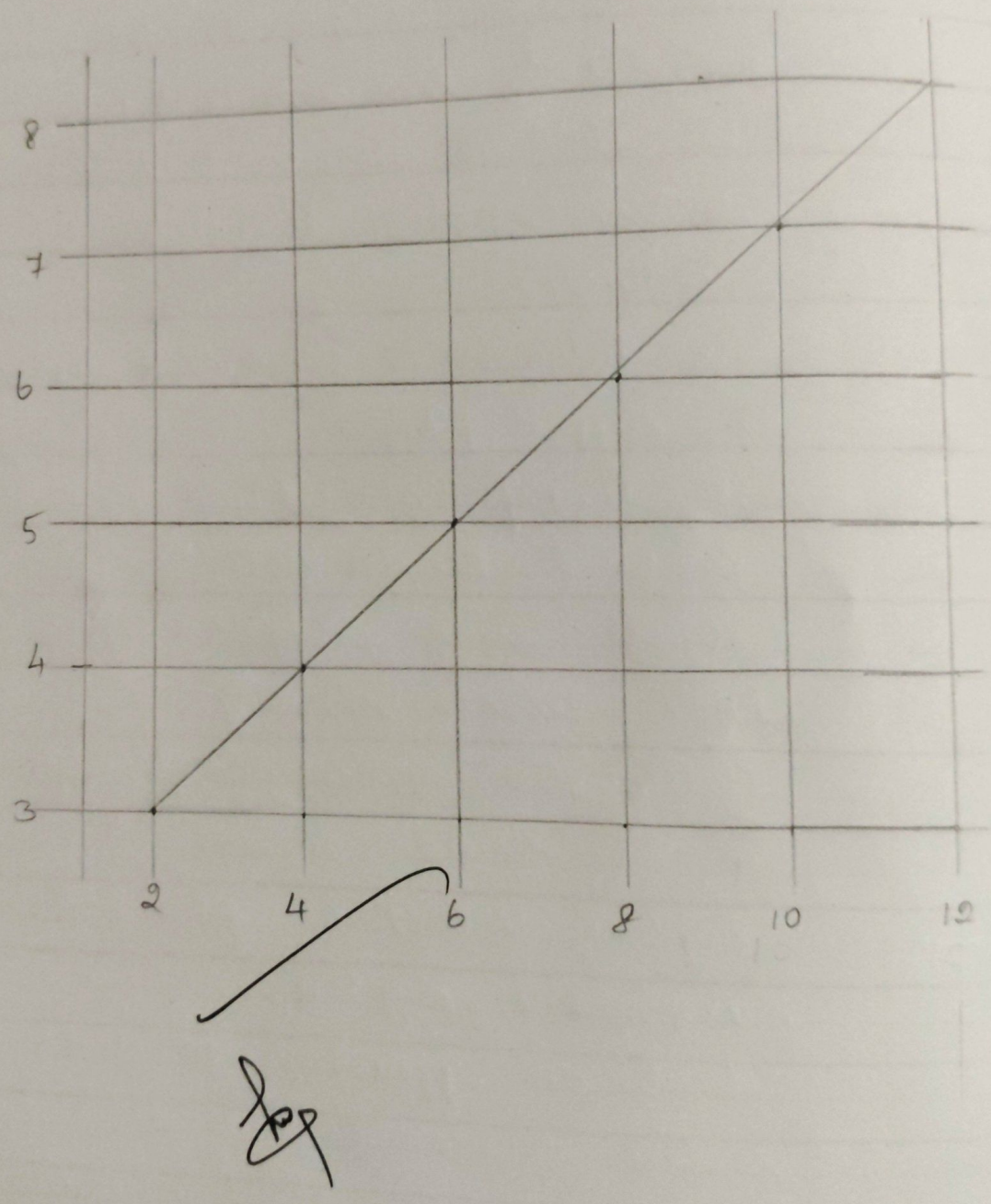
```
import matplotlib.pyplot as plt
x1, y1 = 2, 3
x2, y2 = 12, 8
dx = x2 - x1
dy = y2 - y1
p = 2 * dy - dx
x, y = x1, y1
x, y = [x], [y]
while x < x2:
    x += 1
    if p < 0:
        p += 2 * dy
    else:
        y += 1
        p += 2 * (dy - dx)
    x.append(x)
    y.append(y)
```

```
plt.plot([x[0], x[-1]], [y[0], y[-1]], 'o-')
plt.grid()
plt.show()
```

31/7/24

Result:

Thus the bresenham's line algorithm has been executed successfully.


8
7
6
5
4
3
2
4
6
8
10
12

Expt. No: 1.2

Bresenham's Algorithm for Circle & Elipse.

Aim:

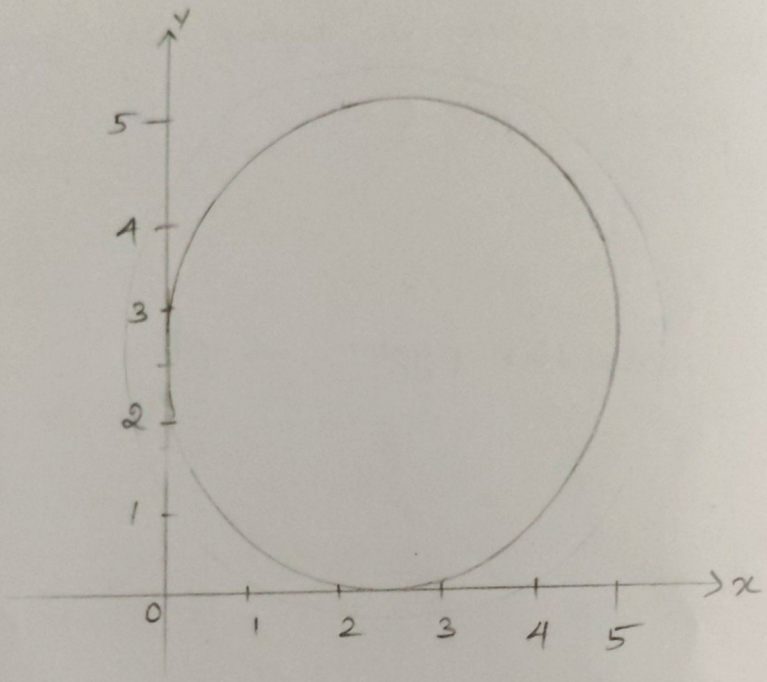
To implement Bresenham's Circle Algorithm & Elipse drawing Algorithm in python & display the generated circle or elipse using Matplotlib.

Program:

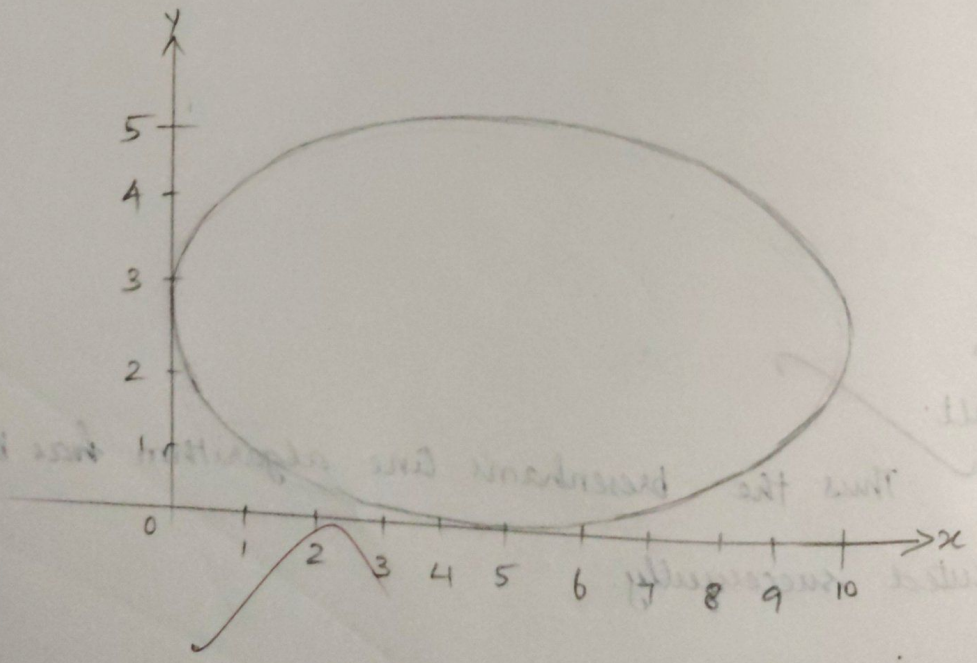
```
import matplot.pyplot as plt
p=[]
def sym(x,y,e=false):
    pts = [(x,y),(-x,y),(x,-y),(-x,-y)]
    p.extend (pts+([(y,x),(-y,x),(y,-x),(-y,x)] if e else[]))

if int(input("1. circle 2. Elipse: "))==1:
    r=100
    x,y,d=0,r,3-2*r
    while x<=y:
        sym(x,y,True)
        if d<0:
            d+=4*x+6
        else:
            d+=4*(x-y)+10
            y-=1
        x+=1
else:
    rx,ry = 120,80
    x,y=0,ry
```

choice 1: circle



Choice 2: Elipse



```
d = xy**2 * ry**2 - 2*rx**2*rx**2*ry**2 + 0.25*rx**2*rx**2
while 2*ry**2*x <= 2*rx**2*y:
    sym(x,y)
    if (d<0):
        d += 2*ry**2*x + ry**2*ry**2
    else:
        y -= 1
        d += 2*ry**2*x - 2*rx**2*rx**2*y + ry**2*ry**2
    x += 1
d = ry**2*(x+0.5)**2 + rx**2*(y-1)**2 - rx**2*rx**2*ry**2
while y>=0:
    sym(x,y)
    if (d>0):
        d -= 2*rx**2*rx**2*y + rx**2*rx**2
    else:
        x += 1
        d += 2*ry**2*x - 2*rx**2*rx**2*y + rx**2*rx**2
    y -= 1
x,y = zip(*set(p))
plt.scatter(x,y,s=5,c='b')
plt.axis("equal")
plt.axis("off")
plt.show()
```

Result:

Thus the Bresenham's Algorithm for circle & elipse has been implemented successfully.

Expt no:

Date : 14/8/26

Bresenham's Algorithm for rectangle

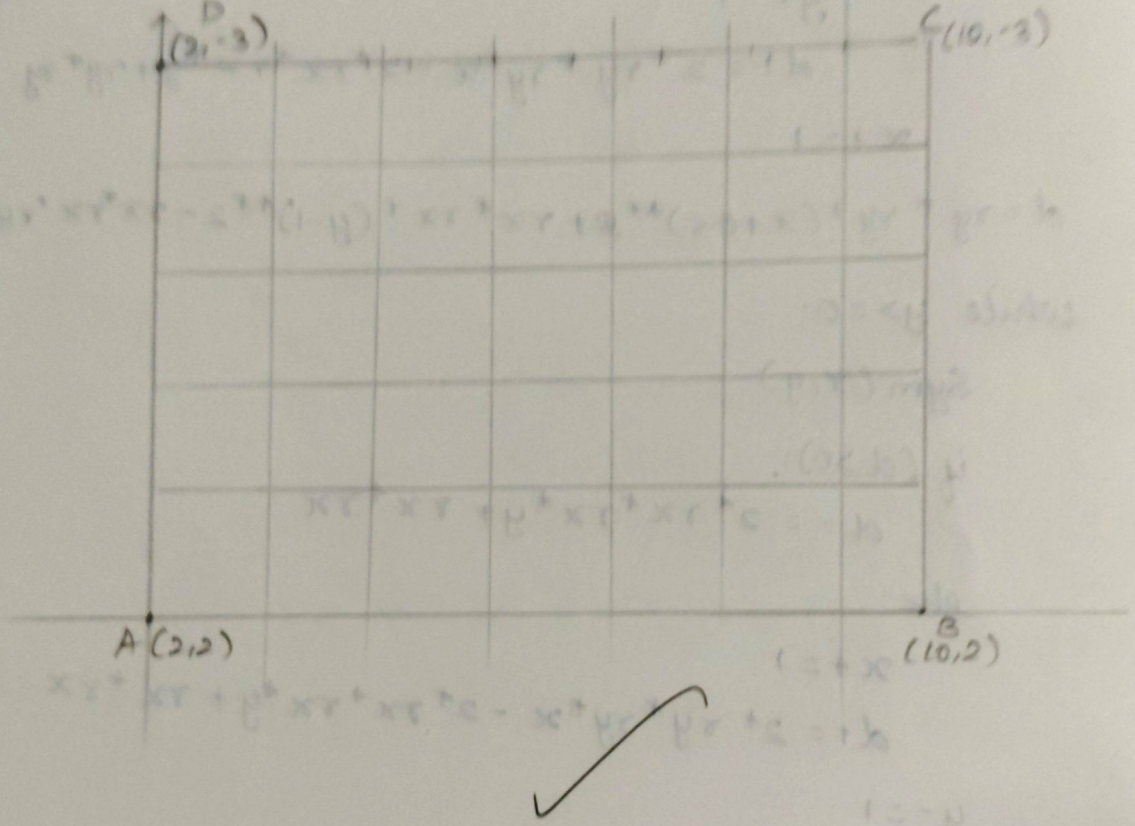
Aim

To implement Bresenham's line Drawing Algorithm in python & use it to draw a rectangle by connecting four edges.

Program:

```
import matplotlib.pyplot as plt

def line(x1, x2, y1, y2):
    p = []
    dx, dy = abs(x2-x1), abs(y2-y1)
    sx, sy = (1 if x2>x1 else -1), (1 if y2>y1 else -1)
    x, y = x1, y1
    if dx>=dy:
        e = 2*dy-dx
        while x!=x2:
            p.append((x,y)); x+=sx
            if e>=0: y+=sy; e-=2*dx
            e+=2*dy
    else:
        e=2*dx-dy
        while y!=y2:
            p.append((x,y)); y+=sy
            if e>=0: x+=sx; e-2*dy
            e+=2*dx
    p.append((x2,y2))
    return p
```


D
(2,-3)
C
(10,-3)
A
(2,2)
B
(10,2)

```
A,B,C,D =
p = line(*A,*B)[:-1]+line(*B,*C)[:-1]+line(*C,*D)[:-1]+
                line(*D,*A)

x,y = zip(*p)
plt.plot(x,y,'-o')
plt.grid()
plt.axis('equal')
plt.show()
```

14/8/26

Result:

Thus the Bresenham's Algorithm for Rectangle using Python has been executed successfully.

Expt. no:

Date: 14/8/26

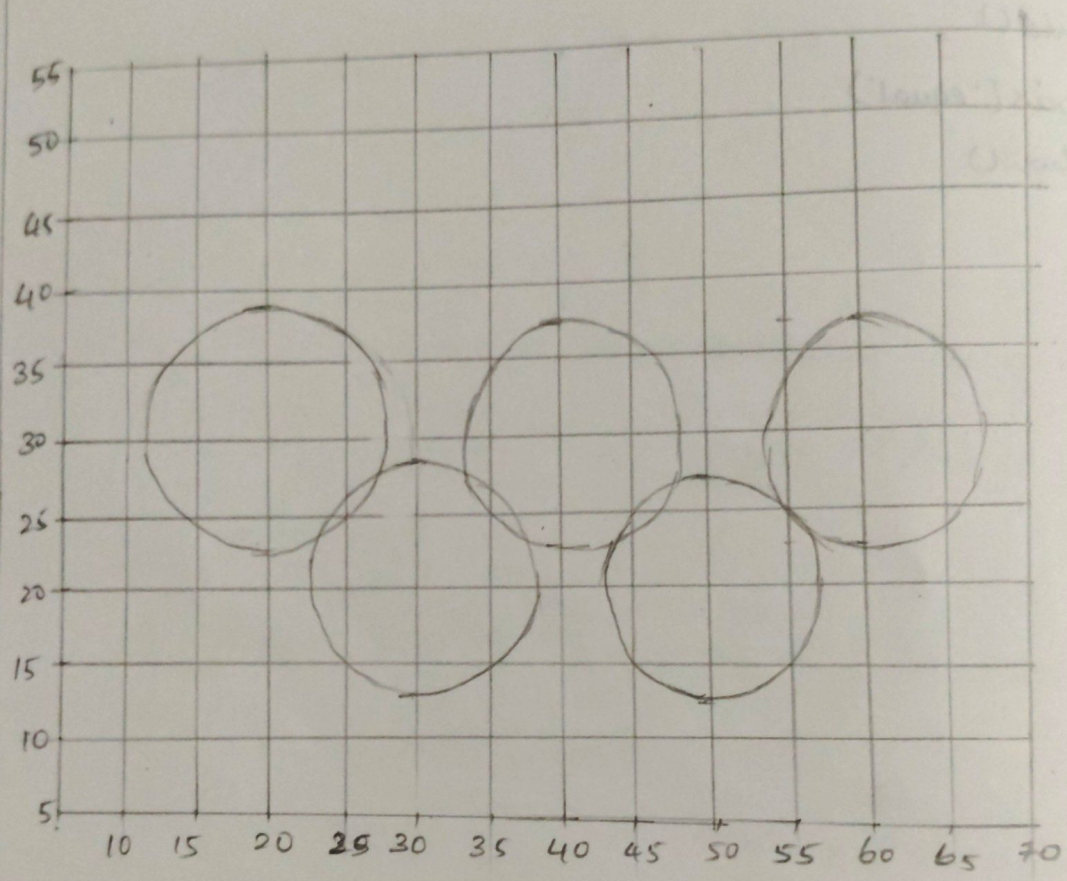
Olympic logo Using midpoint Algorithm

Aim:

To implement the Midpoint Algorithm in python & draw Olympic logo.

Program:

```
import matplotlib.pyplot as plt
def ellipse(rx, ry, xc, yc):
    p = []; x, y = 0, ry
    p = ry*ry - rx*rx*ry + 0.25*rx*rx
    while 2*ry*ry*x < 2*rx*rx*y:
        p += [(xc+x, yc+y), (xc-x, yc+y),
              (xc-x, yc-y), (xc+x, yc-y)]
        if p<0:
            x += 1; y
            p += 2*ry*ry*x + ry*ry
        else:
            x+=1; y-=1
            p += 2*ry*ry*x - 2*rx*rx*y + ry*ry
    p = ry*ry*(x+.5)**2 + rx*rx*(y-1)**2 - rx*rx*ry*ry
    while y>=0:
        p += [(xc+x, yc+y), (xc-x, yc+y),
              (xc-x, yc-y), (xc+x, yc-y)]
        if p>0:
            y-=1; p-=2*rx*rx*y - rx*rx
```


55
50
45
40
35
30
25
20
15
10
5
10 15 20 25 30 35 40 45 50 55 60 65 70

```
        else:
            x+=1 ; y-=1
            p+= 2*ry*ry*x - 2*rx*rx*y + rx*rx + ry*ry
    return zip(*p)

def ring(x,y,c):
    a,b = ellipse(40,40,x,y)
    plt.plot(a,b,c, linewidth=5)

plt.figure(figsize=(10,6))
for x,y,c in [(60,90,'b'),(150,90,'k'),(240,90,'r'),
              (105,45,'gold'),(195,45,'g')]:
    ring(x,y,c)

plt.axis('equal')
plt.axis('off')
plt.show()
```

14/8/26

Result:

Thus the Midpoint Algorithm to draw olympic logo using ~~Python~~ ellipse in python has been implemented successfully.

Expt.no:

Date : 14/8/26

Butterfly using Midpoint Algorithm.

Aim.

To draw Butterfly using ellipse with Midpoint Algorithm.

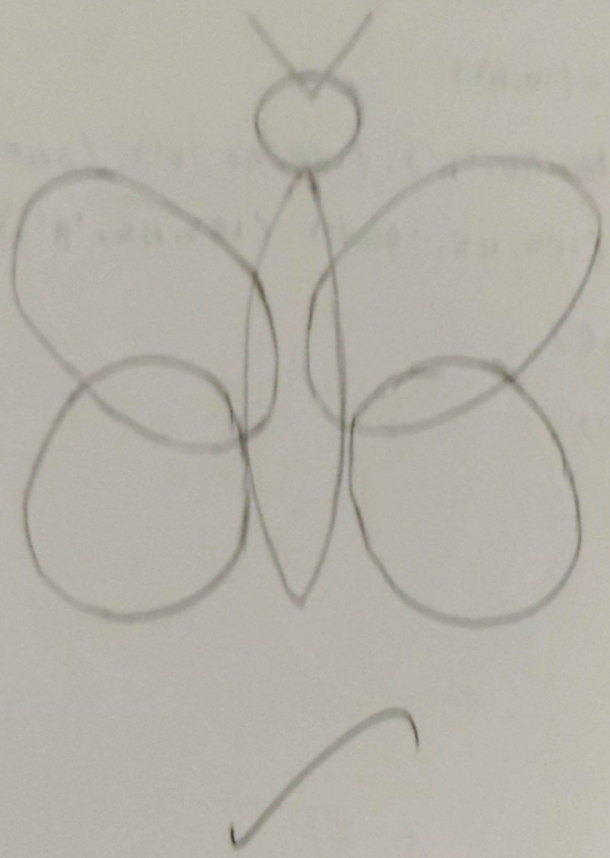
Program:

```
import matplotlib.pyplot as plt
import numpy as np

def ellipse (cx, cy, rx, ry, angle, color):
    rx = abs(rx)
    ry = abs(ry)
    t = np.linspace(0, 2*np.pi, 300)
    x_coords = rx* np.cos(t)
    y_coords = ry* np.sin(t)
    xr = x_coords * np.cos(angle) - y_coords * np.sin(angle)
    yr = x_coords * np.sin(angle) + y_coords * np.cos(angle)
    plt.plot(xr+cx, yr+cy, color = color, linewidth=2)

ellipse(-3, 2, 2.5, 4, np.radians(30), 'blue')
Ellipse (-3, -2, 2, 3, np.radians(-30), 'cyan')
ellipse (3, 2, 2.5, 4, np.radians(-30), 'red')
ellipse(3, -2, 2, 3, np.radians(30), 'magenta')

ellipse (0, 0, 0.5, 5, 0, 'black')
ellipse (0, 5.8, 0.7, 0.8, 0, 'black')
```



```
plt.plot([0,-1], [6.2, 7.5], color='black', linewidth=1.5)
plt.plot([0,1], [6.2, 7.5], color='black', linewidth=1.5)
plt.axis('equal')
plt.axis('off')
plt.show()
```

14/8/21

Result:

Thus the Butterfly using midpoint Algorithm in python has been implemented successfully.

Expt.no:

Date: 14/8/26.

Smiling Face using midpoint Algorithm.

Aim:

To draw a Smiling face using midpoint Algorithm in python.

program:

```
import matplotlib.pyplot as plt
import numpy as np

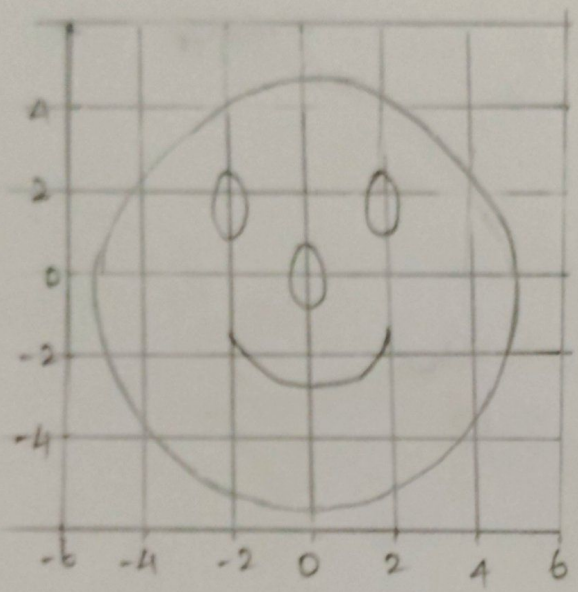
def e(cx, cy, rx, ry, c):
    t = np.linspace(0, 2*np.pi, 100)
    plt.plot(cx + rx*np.cos(t), cy + rx*np.sin(t), c)

def smile(cx, cy, rx, ry):
    t = np.linspace(np.pi, 2*np.pi, 100)
    plt.plot(cx+rx*np.cos(t), cy+ry*np.sin(t), 'r')

plt.figure(figsize=(6,6))
e(0, 0, 5, 5, 'orange')
e(-2, 1.5, .5, .8, 'b')
e(2, 1.5, .5, .8, 'b')
e(0, 0, .4, .8, 'g')

smile(0, -1.5, 2, 1)

plt.axis('equal')
plt.xlim(-6, 6)
plt.ylim(-6, 6)
```


4
2
0
-2
-4
-6
-4
-2
0
2
4
6



```
plt.grid(True)
plt.show()
```

14/8/26

Result:

Thus the Smiling face using midpoint Algorithm has been implemented successfully.

Expt. no:

Date:

Convert Horizontal ellipse to Vertical ellipse

Aim:

To implement the Midpoint Ellipse Drawing Algorithm using python and draw horizontal ellipse then convert into Vertical Ellipse.

Program:

```
import matplotlib.pyplot as plt

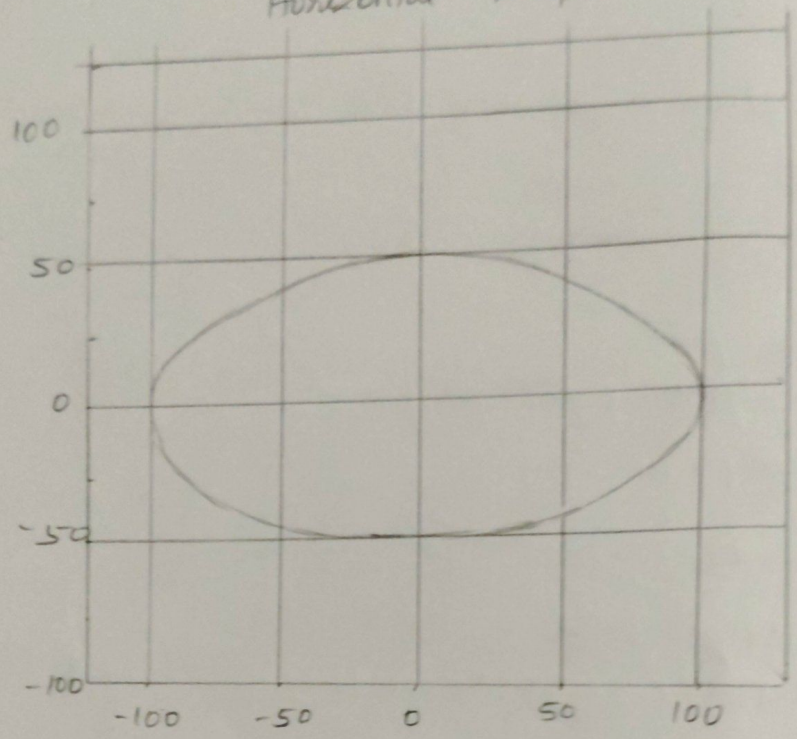
def plot_points(cx, cy, x, y):
    for a, b in [(x,y), (-x,y), (x,-y), (-x,-y)]:
        plt.plot(cx+b, 'b.', markersize=1)

def ellipse(cx, cy, rx, ry):
    x, y = 0, ry
    rx2, ry2 = rx**2, ry**2

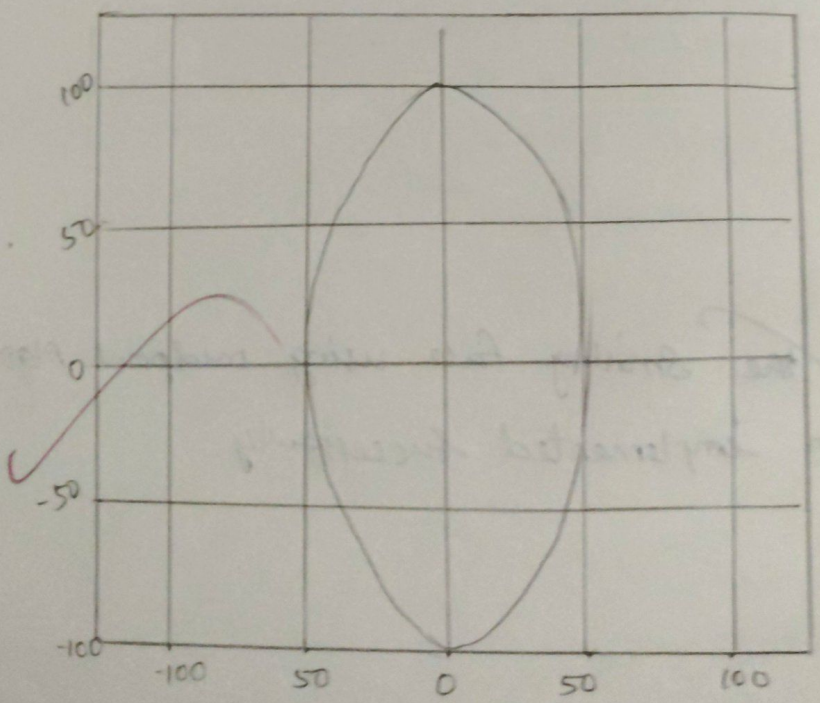
    while 2*ry2*x < 2*rx2*y:
        plot_points(cx, cy, x, y)
        if p<0:
            x+=1
            p+= 2*ry2*x + ry2
        else:
            x+=1
            y-=1
            p+= 2*ry2*x - 2*rx2*y + ry2

    P = ry2*(x+0.5)**2 + rx2*(y-1)**2 - rx2*ry2
```

Horizontal Ellipse.



Vertical Ellipse



```
while y>=0:
    plot_points(cx,cy,x,y)
    if p>0:
        y-=1
        p+=2*a*a-2*a*a*y
    else:
        x+=1
        y-=1
        p+=2*b*b*x-2*a*a*y+a*a
plt.figure(figsize=(10,5))
plt.subplot(1,2,1)
ellipse(0,0,100,50)
plt.title("Horizontal Ellipse")
plt.axis('equal')
plt.grid()
plt.subplot(1,2,2)
ellipse(0,0,50,100)
plt.title("Vertical Ellipse")
plt.axis("equal")
plt.grid()
plt.show()
```

11/9/21

Result:

The Horizontal & vertical ellipse were successfully drawn using the Midpoint Ellipse Algorithm.

Expt.no :

Date :

flood filling Algorithm.

Aim:

Implement flood filling Algorithm to change the colors.

Program:

```
def flood_fill(image, x, y, old_color, new_color):
    if old_color == new_color:
        return
    if x<0 or x>=len(image) or y<0 or y>=len(image[0]):
        return
    if image[x][y] != old_color:
        return
    image[x][y] = new_color
    #up
    flood_fill(image, x-1, y, old_color, new_color)
    #Down
    flood_fill(image, x+1, y, old_color, new_color)
    #left
    flood_fill(image, x, y-1, old_color, new_color)
    #right
    flood_fill(image, x, y+1, old_color, new_color)
```

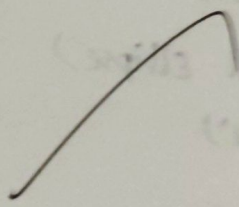
output :

[0, 0, 0, 0, 0]

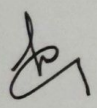
[0, 2, 2, 2, 0]

[0, 2, 2, 2, 0]

[0, 0, 0, 0, 0]



```
image = [
        [0,0,0,0,0],
        [0,1,1,1,0],
        [0,1,1,1,0],
        [0,0,0,0,0]
       ]
flood_fill(image,1,1,2)
for row in image:
      print(row)
```



Result:

Thus the flood-filling Algorithm has been executed successfully.

Expt no:

Date :

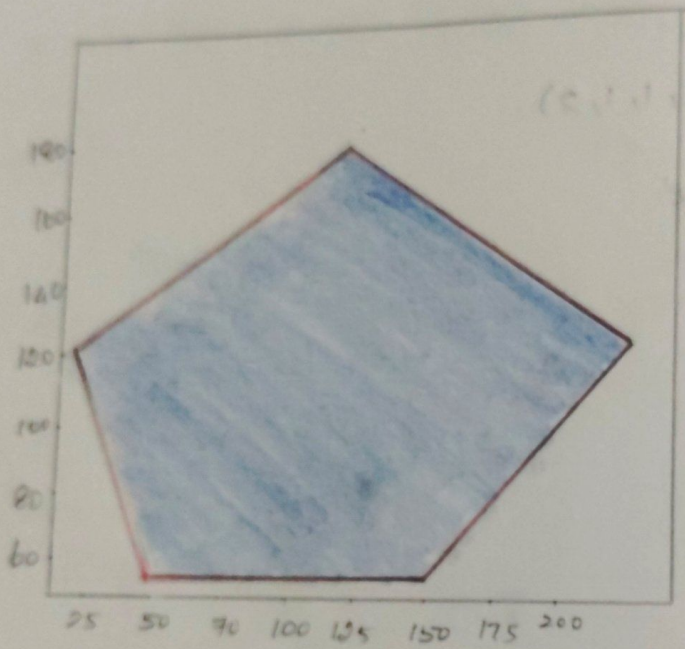
Scan Line Filling Algorithm

Aim:

To implement Midpoint Algorithm to draw a polygon and Scan-Line for Colouring.

Program:-

```
Import matplotlib.pyplot as plt
def line(x1, y1, x2, y2):
    p=[]; dx = abs(x2-x1); dy = abs(y2-y1)
    sx = 1 if x2>x1 else -1; sy=1 if y2>y1 else -1
    x,y = x1,y1
    if dx>dy:
        d = 2*dy - dx
        for i in range(dx+1):
            p.append((x,y)); x+=sx
            if d>=0: y+=sy; d-2*dy
            d+=2*dy
    else:
        d = 2*dx - dy
        for i in range(dy+1):
            p.append((x,y)); y+=sy
            if d>=0: x+sx; d-2*dy
            d+=2*dx
    return p
```


25
50
90
100
150
175
200

```
P = [(50,50),(150,50),(180,120),(100,180),(30,120)]
B = []
for i in range(5):
    Br = line((*p[i], *p[(i+1)%5]))
F = []
for y in range(50,181):
    x = []
    for i in range(5):
        x1,y1 = P[i]; x2,y2 = P[(i+1)%5]
        if (y1<=y<y2) or (y2<=y<y1):
            x.append(int(x1+(y-y1)*(x2-x1)/(y2-y1)))
    x.sort()
    for i in range(0,len(x),2):
        F += [(x,y) for x in range(x[i],x[i+1]+1)]
plt.scatter(*zip(*F),s=5)
plt.scatter(*zip(*B),s=5)
plt.axis('equal'); plt.show()
```

Result:

Thus the midpoint & scan line Algorithm has been implemented successfully.

Ex.no:

Date:

2D Transformation.

Aim:

to write a python program to perform a 2D Transformation for rectangle.

Program:

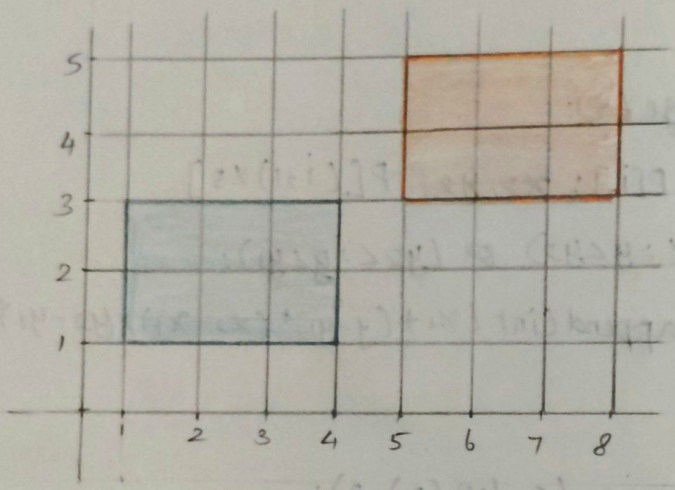
```
import numpy as np
import matplotlib.pyplot as plt
p = np.array([[1,1],[4,1],[4,3],[1,3],[1,1]])
print("1. Translation \n 2. Scaling \n 3. Rotation")
c = int(input("Enter a choice: "))
if c==1:
    x,y = map(float, input("Enter tx ty: ").split())
    q = p + [x,y]
elif c==2:
    x,y = map(float, input("Enter sx sy: ").split())
    q = p * [x,y]
elif :
    a = np.radians(float(input("Enter angle: ")))
    R = [[np.cos(a), -np.sin(a)], [np.sin(a), np.cos(a)]]
    q = p @ np.array(R).T
```

output

1. Translation
2. Scaling
3. Rotation

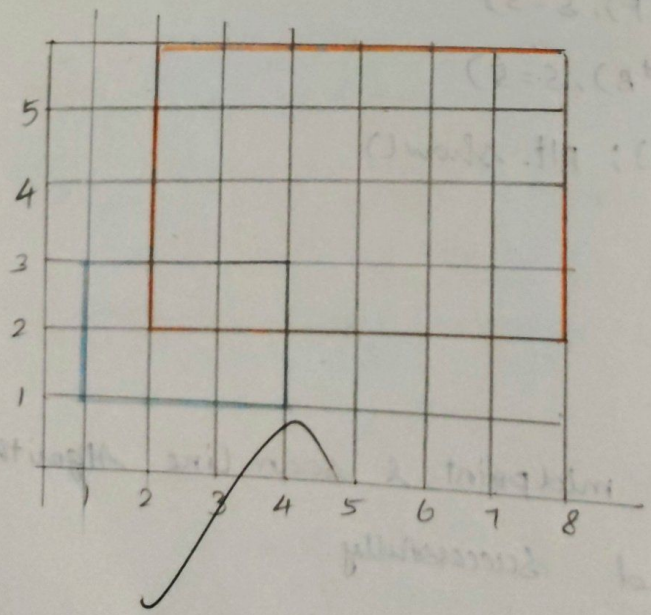
Enter a choice: 1

Enter tx ty: 4 2



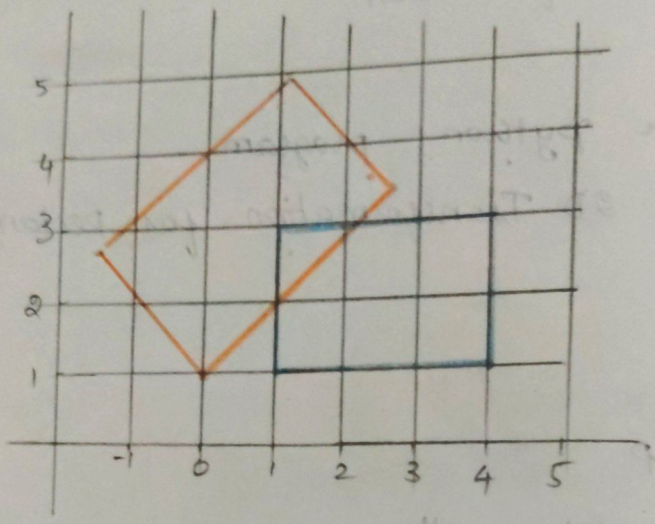
Enter a choice: 2

Enter sx sy: 2 2

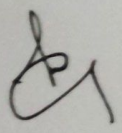


Enter a choice: 3

Enter angle: 45



```
plt.plot(p[:,0],p[:,1], label = "original")
plt.plot(q[:,0],q[:,1], label = "Transformed")
plt.legend()
plt.grid()
plt.axis('equal')
plt.show
```



Result:

Thus the 2D Transformation has been executed successfully.

Ex.no:

Date:

Polygon Clipping

Aim:

To write a python program to implement polygon clipping.

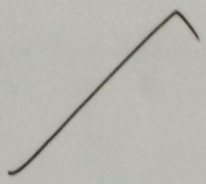
program:-

```
def clip(poly, xmin, ymin, xmax, ymax):
    for s,v in [(0, xmin), (1,xmax), (2,ymin), (3,
                ymax)]:
        q = []
        for i in range(len(P)):
            a,b = p[i-1], p[i]
            ia = (a[0]>=v if s==0 else a[0]<=v if
                s==1 else a[1]>=v if s==2 else
                a[1]<=v)
            ib = (b[0] >=v if s==0 else b[0]<=v
                if s==1 else b[1] >= v if s==2
                else b[1]<=v)
            if ia != ib:
                if s<2:
                    x = v
                    y = a[1] +(b[1]-a[1]*(v-a[0])/
                                (b[0] - a[0])
                else:
                    y = v
                    x = a[0]+ (b[0]-a[0])*(v-a[1])/
                                (b[1]-a[1])
                q.append((x,y))
```

output:

clipping points:

[(3.0, 5), (3.0, 2), (6.0, 2), (6.0, 5)]



```
            if ib:
                q.append(b)

        p=q
    return p
p= [(1,1),(7,1),(7,6),(1,6)]
print (clip(p,3,2,6,5))
```

Result:

Thus the polygon clipping algorithm has been executed successfully.

Expt. no:

Date :

3D Transformation

Aim:-

Write a python code to implement a 3D transformation.

Program:-

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d.art3d import Line3DCollection
v = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],[0,0,1],[1,0,1],
              [1,1,1],[0,1,1]])

e = [[0,1],[1,2],[2,3],[3,0],[4,5],[5,6],[6,7],[7,4],[0,4],[1,5],
     [2,6],[3,7]]

T = np.array([[1,0,0,2],[0,1,0,2],[0,0,1,0],[0,0,0,1]])

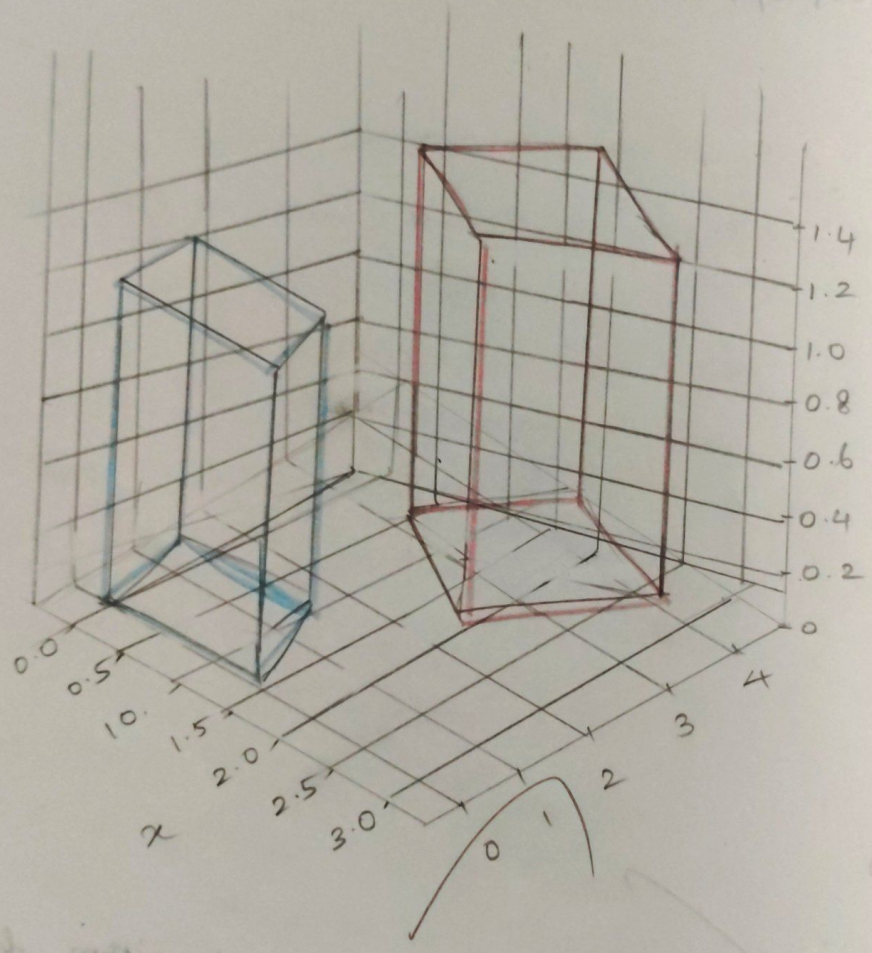
s = np.diag([1.5,1.5,1.5,1])

a = np.radians(45)

R = np.array([[np.cos(a),-np.sin(a),0,0],
              [np.sin(a),np.cos(a),0,0],
              [0,0,1,0],[0,0,0,1]])

V1 = np.array([(T @ s @ R @ np.append(p,1))[:3] for p in v])

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
for points, color in [(v,'blue'),(V1,'red')]:
    ax.add_collection3d(Line3DCollection([[points[i],
        points[j]] for i,j in e], colors=color))
```


1.4
1.2
1.0
0.8
0.6
0.4
0.2
0
0.0
0.5
1.0
1.5
2.0
2.5
3.0
x
0
1
2
3
4

```
ax.set_title("3D Transformation of Cube")
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_zlabel("z")
plt.show
```

Result:

Thus the 3D Transformation for Cube has been implemented.

Expt. no:

Date:

3D object Rendering with flat shading & Basic colors.

Aim:

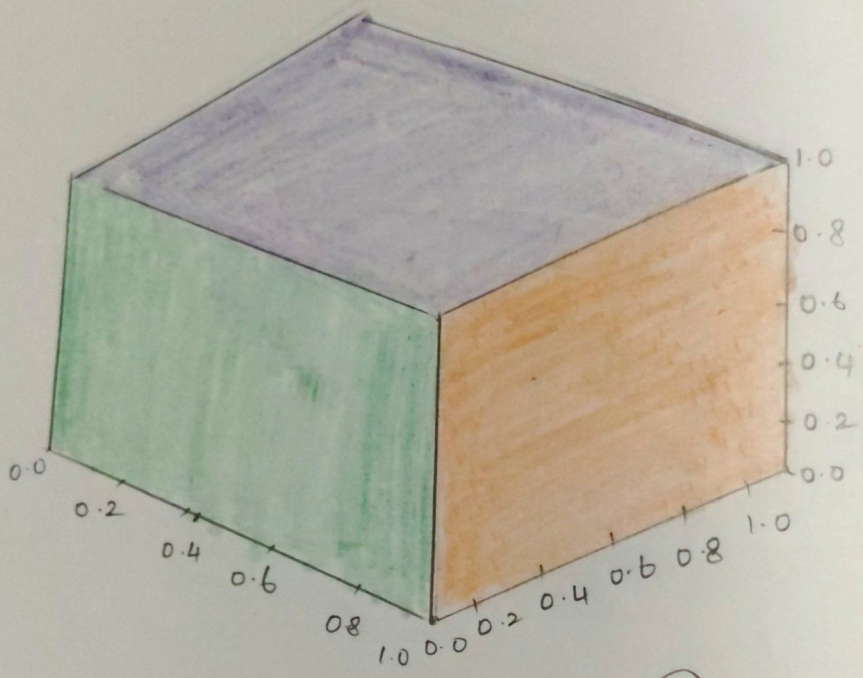
To Create a Rendering with flat a 3D cube using polygonal modelling & apply flat shading & Basic colors.

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkit.mplot3d.art3d import poly3DCollection
vertices = np.array([[0,0,0],[1,0,0],[1,1,0],[0,1,0],
           [0,0,1],[1,0,1],[1,1,1],[0,1,1]])
faces = [[vertices[j] for j in [0,1,2,3]],[vertices[j] for j in
         [4,5,6,7]], vertices[j]for j in [0,1,5,4]],
         [vertices[j] for j in [1,2,6,5]],
         [vertices[j] for j in [4,7,3,0]]]
```

Result:

3D polygonal objects were successfully created & rendered.


1.0
0.8
0.6
0.4
0.2
0.0
0.0
0.2
0.4
0.6
0.8
1.0
0.0
0.2
0.4
0.6
0.8
1.0

Expt. no:

Date:

Create a Multimedia Application that Integrates Images, sound & video in a simple User Interface.

Aim:-

To develop a simple multimedia application that integrate images, sound, video.

Program:

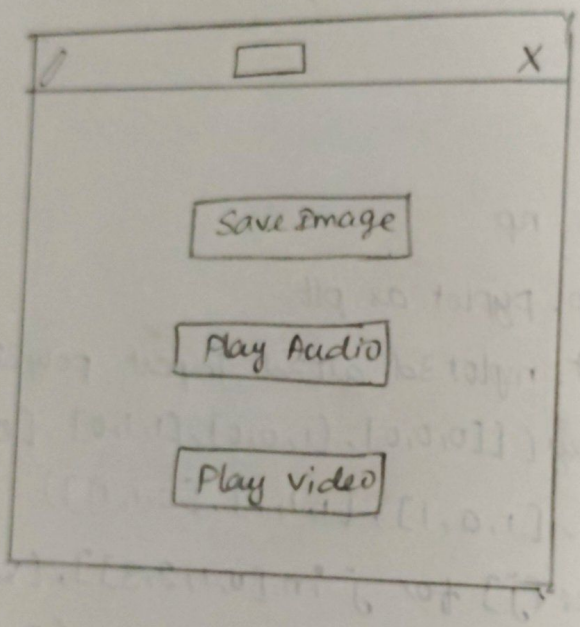
```
import tkinter as tk
from PIL import Image, ImageTk
from playsound import playsound
import cv2, threading.

w = tk.Tk()
w.title("Multimedia App")

def image():
    img = ImageTk.PhotoImage(Image.open("sample_image.jpg").
                             resize((200,200)))
    p.config(image= img)
    p.image = img.

def audio():
    threading.Thread(target = playsound,
                     args = ("sample_audio.mp3",)).start()

def video():
    c = cv2.VideoCapture("sample_video.mp4")
    while True:
        r, f = c.read()
```


X
Save Image
Play Audio
Play Video

```
        if not r: break
        cv2.imshow("video", f)
        if cv2.waitKey(25)==ord('q'): break
    c.release()
    cv2.destroyAllWindows()

p=tk.Label(w)
p.pack()
for t,f in [("Image",image),("Audio",audio),("Video",video)]:
    tk.Button(w,text=t,command=f).pack()

w.mainloop().
```



Result:-

The Multimedia application was successfully created to display images & play audio & video.

Expt. No:

Date:

Capture Video/Audio from Webcam or Microphone & Display on Multimedia Interface.

Aim:

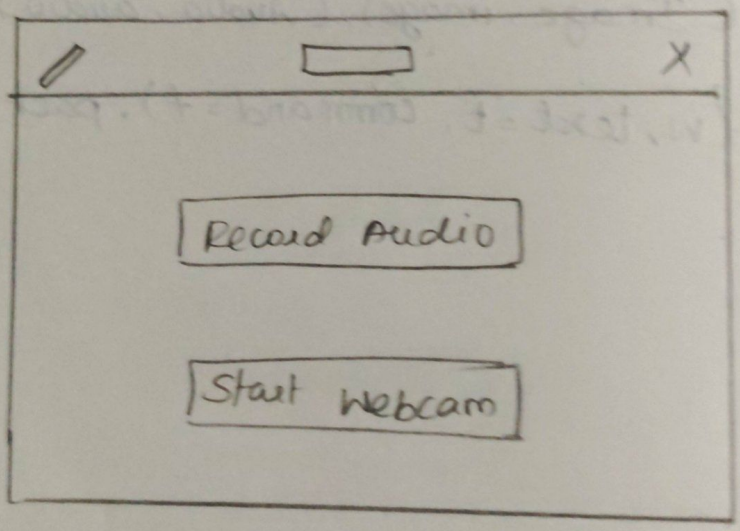
To record audio & capture video using a microphone & webcam & Display.

Program:

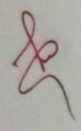
```
import cv2
import sounddevice as sd
from scipy.io.wavfile import write
import tkinter as tk
import threading

def audio():
    fs = 44100
    data = sd.rec(5*fs, samplerate=fs, channels=2)
    sd.wait()
    write("audio.wav", fs, data)

def video():
    c = cv2.VideoCapture(0)
    while True:
        r, f = c.read()
        if not r: break
        cv2.imshow("Webcam", f)
        if cv2.waitKey(1) == ord('q'): break
    c.release()
    cv2.destroyAllWindows()
```


Record Audio
Start Webcam

```
w = tk.TK()
w.title("Audio Video Capture")
tk.Button(w, text = "Record Audio", command = lambda:
        threading.Thread(target = audio).start()).Pack()
ack()
tk.Button(w, text = "start Webcam", command = lambda:
threading.Thread(target = video).start()).P
ack()
w.mainloop()
```



Result:

Audio was recorded & saved as a WAV file, & live Video was captured using the webcam.

Expt. no:

Date:

Create an Application to send & Receive Multimedia Messages (Text, Image, Audio)

Aim:

To Create an application to send text, image, & audio messages using Tkinter.

Program:

```
import tkinter as tk
from tkinter import filedialog
from PIL import Image, ImageTk
from playsound import playsound
from Threading import Thread

w = tk.Tk()
w.title("Multimedia chat")

def send(t):
    p = filedialog.askopenfilename()
    if t = "Text":
        chat.insert(tk.END, "YOU: "+e.get()+"\n")
        e.delete(0, tk.END)
    elif p and t == "Image":
        i = ImageTk.PhotoImage(Image.open(p).resize((100,100)))
        label.config(image = i)
        label.image = i
    elif p:
        Thread(target = playsound, args = (p, )).start()

chat = tk.Text(w, height = 10, width = 30)
chat.pack()
e = tk.Entry(w)
```

```
label = tk.Label(w)
label.pack()
for t in ["Text", "Image", "Audio"]:
    tk.Button(w, text="send" + t,
              command = lambda x=t: send(x)).pack()
w.mainloop().
```

Result:

The application was created successfully to send text, display images, & play audio messages.

Expt. no:

Date:

Create a 3D Model using Blender & Apply Basic shading & Texturing.

aim:

To Create a 3D Model Using Blender with Basic shading & Textures.

Procedure:

1. open Blender & add Basic shapes using Add → Mesh.
2. Combine shapes to create a house or other object.
3. Use Edit Mode to modify the shapes.
4. Apply materials and adjust color & roughness.
5. Add images textures using the shader Editor & UV editing.
6. Add lighting & position the camera.
7. Select Eevee or Cycles and Render the model.

Result:

A model was successfully created in Blender with basic shading, texturing, lighting & rendering.

output

